

รายงานโครงการ CP413008: ทำนายการยกเลิกการจองโรงแรมและแนะนำระดับ Overbooking

บทสรุป

ระบบทำนายความน่าจะเป็นที่การจองโรงแรมจะถูกยกเลิก แล้วแนะนำว่าควรรับจองเกินกี่ห้องต่อคืน บนข้อมูล production 5 เดือน ระบบทำกำไรมากกว่าการไม่รับจองเกิน 27.9% โดยย้ายลูกค้าไปโรงแรมอื่นเพียง 27 คน

- โมเดล: Random Forest + isotonic calibration ได้ PR-AUC 0.752 บน test set สูงเป็น 2 เท่าของ baseline (0.377) และ ECE 0.038 ผ่านเกณฑ์ ≤ 0.05
- การตัดสินใจ: ตีกว่ารับเกินคงที่ 5% ทั้งกำไรและจำนวนลูกค้าที่ถูกย้าย
- ระบบ: pipeline คำสั่งเดียวจบใน 3 นาที 44 วินาที , API ผ่าน SLO ทั้งหมด, มี drift monitoring, retrain อัตโนมัติ และ rollback

กลุ่มวอดส์ · Repository: [Nipoon-T/MLOps-Predict-hotel-bookings-cancellations-and-recommend-an-overbooking-level](#)

รหัสนักศึกษา	ชื่อ	หน้าที่
673380311-7	นางสาวชนินาถ พรหมกันยา	Data & Validation
673380325-6	นางสาวนิตยาพร อินทะชัย	Serving & Load test
673380326-4	นางสาวนิรมล เทศสา	Modeling & Calibration
673380341-8	นางสาวลภัสสรดา วรรณราชู	Pipeline & Registry
673380352-3	นางสาวสุพิชฌาย์ สุทนต์	Decision layer/Backtest
673380523-2	นางสาวนิปูณ ถมปัทม์	Monitoring & CI/CD

1. ปัญหาและคุณค่า

การจองเกือบ 4 ใน 10 ถูกยกเลิก (ราว 37% ของการจองทั้งหมด) แต่โรงแรมไม่รู้ล่วงหน้าว่ารายการไหน โรงแรมจึงต้องเดาว่าควรรับจองเกินกี่ห้องเพื่อชดเชยห้องที่จะว่าง ถ้ารับเกินน้อยไปจะเสียห้องว่าง ถ้ารับเกินมากไปแขกมาครบแต่ไม่มีห้อง

ผู้ใช้ระบบ

ผู้ใช้	ใช้ทำอะไร
ผู้จัดการรายได้ (revenue manager)	ดูจำนวนห้องที่ควรรับจองเกินในแต่ละคืน ผ่าน /recommend-overbooking (batch รายคืน)
ฝ่ายจองของโรงแรม	ดูความน่าจะเป็นที่การจองจะถูกยกเลิกตอนรับจองแต่ละรายการ ผ่าน /predict (real-time)

ผู้ได้รับผลกระทบเมื่อตัดสินใจพลาด

ผู้ได้รับผลกระทบ	รับจองเกินน้อยไป	รับจองเกินมากไป
โรงแรม	ห้องว่าง เสียรายได้	เสียค่าชดเชยและชื่อเสียง
ลูกค้าที่จองไว้	ไม่กระทบ	มาถึงแล้วไม่มีห้อง ถูกย้ายไปโรงแรมอื่น
พนักงานหน้าเคาน์เตอร์	ไม่กระทบ	ต้องรับมือลูกค้าที่ไม่พอใจและหาที่พักใหม่

วิธีเดิมและข้อจำกัด

- ไม่รับจองเกิน (no_overbook): ไม่ต้องย้ายลูกค้า แต่เสียห้องว่างทุกครั้งที่ถูกยกเลิก
- รับเกินคงที่ 5% (fixed): ใช้ตัวเลขเดียวกันทุกคืน ไม่สนว่าคืนนั้นมีการจองเสียยกเลิกมากหรือน้อย
- รับเกินตามอัตรายกเลิกเฉลี่ย (avg_rate): มองทุกการจองเหมือนกันหมด

เหตุผลที่ต้องใช้ ML การจองแต่ละรายการมีโอกาสยกเลิกไม่เท่ากัน ขึ้นกับหลายปัจจัยพร้อมกัน เช่น ระยะจองล่วงหน้า ประเภทมัดจำ ราคา และช่องทางจอง ซึ่งซับซ้อนเกินกว่าจะเขียนเป็นกฎด้วยมือ อีกทั้งพฤติกรรมการยกเลิกเปลี่ยนตามเวลา โมเดลเทรนใหม่จากข้อมูลล่าสุดได้ แต่กฎต้องให้คนกลับมาแก้เอง

2. AI Project Canvas

ช่อง	เนื้อหา
Value proposition	แนะนำจำนวนห้องที่ควรรับจองเกินต่อคืน เพื่อขายห้องที่ถูกยกเลิกได้ โดยย้ายลูกค้าให้น้อยที่สุด
Prediction task	จำแนก is_canceled ให้ผลเป็นความน่าจะเป็นที่ calibrate แล้ว (p_cancel)
Decisions	จำนวนห้องที่รับจองเกินต่อคืนต่อโรงแรม เลือกจาก Monte Carlo ที่ต้นทุนคาดหวังต่ำสุด
Data	Hotel Booking Demand (Kaggle) 119,390 การจอง 32 คอลัมน์ จาก City Hotel และ Resort Hotel ปี 2015–2017 ใช้เฉพาะข้อมูลที่อยู่ตอนจอง
Data collection	label รู้ผลเมื่อถึงวันเข้าพัก (label มาช้า) จึงวัดความแม่นยำได้เฉพาะสัปดาห์ที่ผ่านมาไปแล้ว
Features	lead_time, deposit_type, adr, market_segment ฯลฯ ดัดคอลัมน์ leakage ตาม configs/columns.yaml
Building models	เทรนใหม่บนหน้าต่างข้อมูลล่าสุดเมื่อพบ drift มี cooldown 4 สัปดาห์ และ gate ก่อน promote
Making predictions	real-time ผ่าน /predict ตอนรับจอง และ batch ผ่าน /recommend-overbooking ก่อนตัดสินใจรายคืน
Impact simulation	backtest 4 นโยบายบนข้อมูลจริง วัดกำไรและจำนวนลูกค้าที่ถูกย้าย

ช่อง	เนื้อหา
Monitoring	data drift (ไม่ต้องใช้ label), prior shift และ concept drift (เมื่อ label มาถึง), latency และ error rate
Costs	ห้องว่างเพราะรับจองน้อยไป = เสียค่าห้อง 1 คืน · ห้องไม่พอเพราะรับจองมากไป = จ่ายชดเชย 2 เท่าของค่าห้อง ($k = 2.0$)
Revenue	ขายห้องที่ถูกยกเลิกให้คนอื่นได้ จำลองย้อนหลัง 5 เดือน กำไรเพิ่ม 27.9% เทียบกับไม่รับจองเกิน
Stakeholders	ภายใน: ผู้บริหาร, ผู้จัดการรายได้, ฝ่ายต้อนรับ, ฝ่ายห้องพัก, ฝ่ายขาย, IT, ML · ภายนอก: ลูกค้าที่เข้าพัก, บริษัททัวร์
Skills	Data engineer, ML engineer, revenue management, backend/DevOps

3. ตัวชี้วัด

ใช้ PR-AUC เป็นตัวชี้วัดหลัก เพราะข้อมูลไม่สมดุล (ยกเลิกประมาณ 37%) และสนใจการจัดอันดับความเสี่ยงของเคสยกเลิกมากกว่า accuracy รวม ส่วน gating metric แบ่งเป็น 2 กลุ่ม คือ ด้านของโมเดล ตรวจสอบว่าโมเดลดีพอจะขึ้นใช้งานหรือไม่ก่อน promote และด้านของระบบ ตรวจสอบว่า API ตอบได้เร็วและเสถียรตาม SLO ซึ่งวัดด้วย load test

ประเภท	ตัวชี้วัด	เกณฑ์	ผลที่ได้
Optimizing	PR-AUC บน test set	สูงที่สุด และสูงกว่า baseline 0.377	0.752
Gating (โมเดล)	ECE	≤ 0.05	0.038
Gating (โมเดล)	PR-AUC ลดลงจาก champion	≤ 0.03	ผ่าน
Gating (โมเดล)	กำไรจำลองเทียบ champion	$\geq$ champion	ผ่าน
Gating (ระบบ)	p95 latency ที่ 120 req/s	≤ 350 ms	330 ms
Gating (ระบบ)	Throughput สูงสุด	≥ 150 req/s	158 req/s
Gating (ระบบ)	Error rate	$\leq 1\%$	0%
Business	กำไรจำลองเทียบไม่รับจองเกิน และ จำนวนลูกค้าที่ถูกย้ายตลอดช่วง ทดสอบ	กำไรเพิ่มโดยย้ายลูกค้าน้อย	+27.9%, ย้าย 27 คน

เหตุผลที่ใช้ ECE เป็นเกณฑ์ ชันตัดสินใจไม่ได้ใช้แค่ลำดับความเสี่ยงของการจอง แต่นำค่า p_cancel ไปคำนวณจำนวนลูกค้าที่คาดว่าจะมาเข้าพักจริงโดยตรง ค่าความน่าจะเป็นจึงต้องตรงกับอัตรายกเลิกจริง หากโมเดลประเมินโอกาสยกเลิกต่ำกว่าความจริง ระบบจะแนะนำให้รับจองเกินน้อยเกินไปและเหลือห้องว่าง หากประเมินสูงกว่าความจริง ระบบจะแนะนำให้รับจองเกินมากเกินไปจนต้องย้ายลูกค้า ดังนั้นโมเดลที่มี PR-AUC สูงแต่ ECE ไม่ผ่านเกณฑ์จึงไม่เหมาะสมสำหรับนำไปใช้ตัดสินใจ

4. สถาปัตยกรรมและเครื่องมือ

ระบบรันทั้งสายด้วย Prefect ตั้งแต่ข้อมูลดิบจนถึง API โดยเก็บโมเดลและ alias ไว้ใน MLflow เมื่อ monitoring พบ drift จะวนกลับไปเทรนใหม่

1. ข้อมูลดิบ: ดาวน์โฮลด์จาก Google Drive ของทีม ตรวจสอบ SHA-256
2. Clean: ลบแถวผิดปกติ (ADR ติดลบ, ผู้เข้าพัก 0)
3. Validate + Split: ตรวจสอบ schema บนข้อมูลที่ clean แล้ว ผิดปกติจะหยุด pipeline ผ่านแล้วแบ่งชุดตามวันเข้าพัก
4. Train: เทรน 5 การทดลอง บันทึกด้วย MLflow
5. Calibrate: isotonic regression บน validation set
6. Gate(ในวงจร retrain) : ตรวจสอบ ECE, PR-AUC และถ้าไม่ผ่านจะไม่ promote และ --promote ใช้สำหรับสลับ alias และ redeploy เมื่อผู้ใช้ต้องการ promote candidate ที่ผ่านการตรวจแล้ว
7. Registry: ตั้ง alias candidate / champion / previous_champion
8. Export: ส่งออกโมเดลไปที่ serving_model/
9. Serving: FastAPI ใน Docker 8 workers มี healthcheck
10. Monitoring: /health, /metrics และตรวจสอบ drift รายสัปดาห์
11. Retrain: เมื่อพบ drift และผ่าน cooldown 4 สัปดาห์ เทรน + calibrate ใหม่บนหน้าต่างข้อมูลล่าสุด 12/6/3 เดือน แล้วผ่าน gate

ด้าน	เครื่องมือ	เหตุผลที่เลือก
Version control	GitHub	branch และ PR ตอบทบาท ประวัติ commit ใช้ ประเมินรายบุคคล
Containerization	Docker + Compose	API รันเหมือนกันทุกเครื่อง มี healthcheck ในตัว
Data validation	Pandera + validate.py	schema เป็นโค้ด Python ทดสอบใน pytest ได้
Experiment tracking	MLflow 3.16.1	บันทึก run ครบ 6 อย่างในที่เดียวกับ registry
Model registry	MLflow Model Registry	จัดการ alias champion/candidate และ rollback ด้วยการสลับ alias
Pipeline orchestration	Prefect 3.7.2	DAG เป็น Python ธรรมดา มี UI ดู run เวอร์ชันนี้ ไม่ชนกับ fastapi 0.115
Model serving	FastAPI 0.115 + uvicorn	Pydantic ตรวจสอบ input ตอบ 422 อัปเดตโมเดล รองรับ async
Monitoring	Evidently + NannyML + /metrics	ตรวจสอบ drift ได้โดยไม่ต้องใช้ label และติดตาม p50/p95/error
CI/CD	GitHub Actions	รัน ruff + pytest ทุก push และ PR เข้า main
Load test	Locust 2.46.6	เขียน scenario ข้อมูลถูกผลิต 10:1 เป็น Python ได้

เวอร์ชันไลบรารีทั้งหมดล็อกไว้ใน requirements.txt และ requirements-pipeline.txt (เช่น pandas 3.0.6, scikit-learn 1.9.1, lightgbm 4.6.0, pandera 0.33.1) และ requirements-monitoring.txt (NannyML + Evidently บน Python 3.12 เพราะ NannyML ยังไม่รองรับ 3.13)

5. ข้อมูล

ข้อมูลผ่าน 3 ด้านก่อนถึงการเทรน คือตรวจ checksum, clean และ validate gate จาก 119,390 แถวเหลือ 119,209 แถว (ตัดออก 0.15%) แล้วแบ่งชุดตามเวลา

ที่มา: Hotel Booking Demand บน Kaggle แจกในทีมผ่าน Google Drive แล้ว src/download_data.py ตรวจ SHA-256 ทุกครั้ง ไม่ตรงจะหยุดทันที

Validation (src/validate.py + Pandera): ตรวจ 32 คอลัมน์ ชนิดข้อมูล ค่า binary ค่า category วันที่ ค่าติดลบ และจำนวนผู้เข้าพักที่ต้องไม่เป็น 0 บนข้อมูลดิบใช้เป็นรายงาน (พบปัญหา 181 แถวตามที่คาด) ส่วนบนข้อมูลหลัง clean ใช้เป็นค่าจริง ผล PASS

การ clean (รายละเอียดใน data_decisions.md)

ปัญหา	จำนวน	การตัดสินใจ
ADR ติดลบ	1 แถว	ตัดออก
ผู้เข้าพักเป็น 0	180 แถว	ตัดออก
แถวซ้ำ	31,980 แถว (ไม่นับแถวแรก)	ไม่ลบ สร้าง flag is_duplicate
ค่าว่าง country / agent / company	478 / 16,280 / 112,441	เติม Unknown ใน features.py
ค่าว่าง children	4	เติม 0 ใน features.py
ADR สูงสุด 5,400	—	ไม่ตัด เพราะต่ำกว่าเพดาน validation 6,000

ถ้าตัดข้อมูลเกิน 1% clean_data.py จะหยุด pipeline เพื่อกันข้อมูลหายจำนวนมากโดยไม่มีคนตรวจ

การแบ่งชุดตามเวลา (arrival_date)

ชุด	ช่วงเวลา	จำนวนแถว	อัตรายกเลิก
Train	2015-07-01 → 2016-06-30	49,108	35.8%
Validation	2016-07-01 → 2016-12-31	29,482	36.9%
Test	2017-01-01 → 2017-03-31	12,789	33.5%
Production	2017-04-01 → 2017-08-31	27,830	41.2%

แบ่งตามเวลาแทนการสุ่ม เพราะในการใช้งานจริงโมเดลต้องทำนายอนาคตจากอดีต การสุ่มจะทำให้โมเดลเห็นข้อมูลอนาคตระหว่างเทรน ชุด production เก็บไว้จำลองการใช้งานจริงรายสัปดาห์ split_data.py ตรวจลำดับเวลา การซ้อนทับ และบันทึก SHA-256 ของทุกชุดใน split_manifest.json จึงได้ผลเดิมทุกครั้ง

การกัน leakage (configs/columns.yaml) ตัดคอลัมน์ที่รู้ค่าหลังการจอง: reservation_status, reservation_status_date, assigned_room_type, booking_changes, required_car_parking_spaces, total_of_special_requests และไม่ใช้ arrival_date_year เพราะแบ่งชุดตามเวลา ปีใน test และ production จึงเป็นค่าที่ไม่เคยไม่เคยเห็นตอนเทรน

ฟีเจอร์ที่ตรวจเพิ่ม (modeling_notes.md)

- adr: ไม่พบแนวโน้มที่บ่งว่าราคาเปลี่ยนหลังการจอง จึงเก็บไว้ แต่เป็นการตรวจเชิงสถิติ ไม่ใช่การยืนยันจากเจ้าของข้อมูล
- deposit_type: กลุ่ม Non Refund ยกเลิก 99.03% ใน train (7,494 แถว) ส่วน No Deposit 24.46% เป็นไปได้ว่าเป็นความผิดปกติเฉพาะของสองโรงแรมนี้ เก็บไว้เพราะรู้ค่าตอนจองจริง ตัดออกแล้ว PR-AUC ลดจาก 0.785 เหลือ 0.744 (บน test ก่อน calibrate)

6. การทดลองและการเลือกโมเดล

โมเดลที่เลือกคือ Random Forest (max_depth=8, min_samples_leaf=50, n_estimators=300) + isotonic calibration ได้ PR-AUC 0.752 และ ECE 0.038 บน test set ลงทะเบียนเป็น models/hotel-cancellation-classifier

การทดลอง	ชุดที่วัด	PR-AUC	ECE
Baseline (อัตรายกเลิกเฉลี่ยตามโรงแรม + เดือนจาก train)	Validation	0.377	0.093
Logistic Regression	Validation	0.760	0.115
Random Forest	Validation	0.765	0.099
Random Forest (tuned)	Validation	0.768	0.096
LightGBM	Validation	0.741	0.195
RF tuned + isotonic (เลือกใช้)	Test	0.752	0.038

ทำแถวแรกวัดบน validation ส่วนแถวสุดท้ายวัดบน test จึงเทียบกันตรงๆ ไม่ได้ บน test เอง PR-AUC ก่อน calibrate คือ 0.785 และหลัง calibrate คือ 0.752 เพราะ isotonic ทำให้คะแนนเป็นขั้นและมีค่าซ้ำกัน ทำให้อันดับของคะแนนละเอียดน้อยลงเล็กน้อย แต่ทีมยอมแลก เพราะ ECE บน test ลดจาก 0.115 (ก่อน calibrate) เหลือ 0.038 ซึ่งสำคัญกว่าสำหรับขั้นตัดสินใจ ทุกโมเดลมี ECE ก่อน calibrate เกิน 0.05 จึงต้อง calibrate

Calibration ใช้ isotonic regression fit บน validation set แล้วประเมินบน test set

การปรับ hyperparameter RF ชุดแรก (max_depth=12, min_samples_leaf=20) หลัง calibrate ได้ ECE 0.0511 เกินเกณฑ์เล็กน้อย ทีมจึงเพิ่มการ regularize เพราะดันไม่ที่ลึกลงไปให้ความน่าจะเป็นที่มั่นใจเกินไปจริง

max_depth	min_samples_leaf	PR-AUC (test)	ECE (test)	ผล
12	20	0.7600	0.0511	ไม่ผ่าน
10	30	0.7573	0.0474	ผ่าน แต่ margin แคบ
8	50	0.7519	0.0379	เลือกใช้
6	100	0.7363	0.0307	ผ่าน แต่ PR-AUC ลดมาก

เลือก max_depth=8 เพราะ ECE ต่ำกว่าเกณฑ์ 0.012 แลกับ PR-AUC ที่ลดลงเพียง 0.008 จากชุดแรก ข้อควรระวังคือการเลือกค่านี้นี้ดูจาก test set ผลบน test จึงอาจดีกว่าความจริงเล็กน้อย

MLflow บันทึก 6 อย่างทุก run: git SHA, hash ของ train set, hyperparameters, ตัวชี้วัด (PR-AUC, ROC-AUC, log-loss, Brier, ECE), artifacts (โมเดล + reliability curve) และ environment.txt

7. ขั้นตอนตัดสินใจ Overbooking

นโยบาย model ได้กำไรมากกว่าไม่รับจองเกิน 27.9% บน production โดยย้ายลูกค้า 27 คน และชนะรับเกินคงที่ 5% ทั้งกำไรและจำนวนลูกค้าที่ถูกย้าย

วิธีคำนวณ

- 1. รับ p_cancel ที่ calibrate แล้วของทุกการจองในคืนนั้น
- 2. Monte Carlo 2,000 รอบ (seed 42) จำลองว่าแต่ละการจองมาจริงหรือไม่ตามความน่าจะเป็น $1 - p_cancel$
- 3. ใส่จำนวนห้องรับเกิน o ตั้งแต่ 0 ถึง 30% ของความจุ หักห้องที่แขกพักต่อจากคืนก่อน (already_occupied) แล้วเลือก o ที่ต้นทุนคาดหวังต่ำสุด

$$\text{ต้นทุนคาดหวัง}(o) = E[\text{ห้องว่าง}] \times \text{adr} + E[\text{ลูกค้าที่ถูกย้าย}] \times k \times \text{adr}$$

สมมติฐาน ความจุ City Hotel 190 ห้อง และ Resort Hotel 91 ห้อง จาก p95 ของจำนวนห้องที่มีแขกพักจริงต่อคืน (นับการพักหลายคืน) ค่าชดเชยการย้ายลูกค้า $k = 2.0$ เท่าของค่าห้อง เพราะรวมค่าห้องทดแทน ค่าเดินทาง และความเสียหายต่อชื่อเสียง และทดสอบ $k = 1.5$ กับ 3.0 ด้วย

นโยบายเปรียบเทียบ no_overbook = ไม่รับเกิน, fixed = รับเกิน 5% ของความจุทุกคืน, avg_rate = ความจุ $\times$ อัตรายกเลิกเฉลี่ยรายเดือน
ผล backtest แบบพักต่อเนื่องหลายคืน ($k = 2.0$)

นโยบาย	กำไร test (204 คืน-โรงแรม)	ย้ายลูกค้า test	กำไร production (329 คืน-โรงแรม)	ย้ายลูกค้า production	กำไรเทียบไม่รับเกิน (test / production)
avg_rate	1,113,504	1,502	4,031,032	6,521	+40.5% / +36.9%
model	993,129	34	3,767,922	27	+25.3% / +27.9%
fixed 5%	898,073	70	3,436,208	33	+13.3% / +16.7%
no_overbook	792,663	0	2,945,321	0	—

ทำไมเลือก model แทน avg_rate avg_rate ได้กำไรมากกว่า 7% บน production แต่ย้ายลูกค้ามากกว่า model 241 เท่า (44 เท่าบน test) เพราะสูตรไม่นับแขกที่ยังพักต่อจากคืนก่อน จึงรับเกินซ้อนทุกคืน เฉลี่ยเกือบ 20 คนต่อคืนซึ่งใช้จริงไม่ได้ นอกจากนี้ backtest นับต้นทุนการย้ายแค่ $k \times \text{adr}$ ยังไม่รวมริ้วรอยและลูกค้าที่ไม่กลับมา ต้นทุนจริงของ avg_rate จึงสูงกว่าที่เห็น

Sensitivity model ชนะ fixed 5% และ no_overbook ทุกค่า k แต่อันดับกับ avg_rate ขึ้นกับ k เมื่อค่าชดเชยสูงขึ้นไปเป็น 3 เท่าของค่าห้อง model กำไรมากกว่า avg_rate บน production (3,761,027 กับ 3,110,350) เพราะ avg_rate ย้ายลูกค้า 6,521 คนทุกค่า k ส่วน model ย้ายลดลงจาก 29 เหลือ 26 คนตาม k บน test avg_rate ยังกำไรสูงกว่าเล็กน้อยที่ $k = 3.0$ (1,007,173 กับ 989,422) ผลครบบ่อยใน overbooking_decision_result/*_sensitivity.csv

ตัวเลขชุดนี้มาจาก backtest_sequential.py ผลดิบอยู่ที่ overbooking_decision_result/real_backtest_summary.csv (test) และ overbooking_decision_result/prod_sim_backtest_summary.csv (production)

8. Model Registry และ Rollback

ทุกโมเดลลงทะเบียนใน MLflow ชื่อ hotel-cancellation-classifier และใช้ alias 3 ตัวคุมว่าตัวไหนใช้งาน rollback จึงทำได้ด้วยคำสั่งเดียวโดยไม่ต้องแก้โค้ด

alias	ความหมาย
candidate	โมเดลใหม่ล่าสุดจาก calibrate.py ยังไม่ใช้งาน

alias	ความหมาย
champion	โมเดลที่ API ใช้อยู่
previous_champion	champion ตัวก่อนหน้า สำรองไว้ rollback

- `python -m pipeline.run --promote` ย้าย champion เดิมไปเป็น previous_champion แล้วตั้ง candidate เป็น champion
- `python -m pipeline.run --rollback` สลับ champion กับ previous_champion
- ทั้งสองคำสั่ง export โมเดลและรีสตาร์ท API ให้ทันที ตรวจได้จาก `serving_model/EXPORTED_FROM.txt`

หลักฐาน: ก่อน rollback v6 (จาก retrain สัปดาห์ 5) เป็น champion และ v5 เป็น previous_champion หลัง --rollback สลับกัน v5 กลับมาเป็น champion และ API เปลี่ยนตามอัตโนมัติ ภาพหน้าจออยู่ที่ [docs/evidence/registry/](#)

9. Serving และ SLO

API เป็น FastAPI ใน Docker 8 workers ผ่าน SLO ทั้ง 3 ข้อ วัดด้วย Locust บน laptop 20 threads ที่รัน Locust ร่วมกัน

Endpoint	รูปแบบ	ใช้เมื่อ
GET /health	สถานะ	API พร้อมและโหลดโมเดลแล้ว ใช้เป็น Docker healthcheck
GET /metrics	สถานะระบบ	จำนวน request, error rate, p50/p95
POST /predict	Real-time 1 การจอง	ฝ่ายจองรับการจองใหม่
POST /predict-batch	Batch $\leq 1,000$ รายการ	ให้คะแนนการจองทั้งคืน
POST /recommend-overbooking	Batch รายคืน	จำนวนห้องที่ควรรับเกิน ต้องส่ง already_occupied

ใช้ 2 รูปแบบเพราะการประเมินตอนรับจองต้องตอบทันที ส่วนการตัดสินใจ overbooking ทำวันละครั้งต่อคืนเข้าพัก แยก endpoint ทำให้ปรับแต่ละส่วนได้อิสระ

Logs: uvicorn บันทึก access log ของทุก request ดูได้ด้วย `docker compose logs api` (หรือ `docker logs hotel-api`) และตอนเริ่มระบบจะบันทึกการโหลดโมเดลของแต่ละ worker ส่วน /metrics สรุปจำนวน request, error rate และ p50/p95

SLO	เป้า	วัดได้	ผล
Throughput ต่อ container	≥ 150 req/s	158 req/s	ผ่าน
p95 ของ /predict ที่ 120 req/s	≤ 350 ms	330 ms	ผ่าน
p50 ของ /predict ที่ 120 req/s	ไม่ตั้ง SLO	130 ms (ไม่มีคิว 28 ms)	—
Error rate (5xx)	$\leq 1\%$	0%	ผ่าน

SLO เดิมตั้งไว้ 200 req/s และ $p95 \leq 100$ ms ทำไม่ได้บน laptop จึงปรับตามที่วัดจริง ผลดิบทุกรอบอยู่ใน `reports/load_test/`

การปรับปรุงระหว่างวัด

การปรับ	ก่อน	หลัง
บังคับโมเดลทำนายแบบ thread เดียว (เทรนด้วย n_jobs=-1)	39.1 ms	18.7 ms
8 workers แทน 1 (Windows, 20 ผู้ใช้)	30 req/s	166 req/s
เปิด TCP_NODELAY ให้ทุก worker ใน Docker	132 req/s	158 req/s
เวลาตอบกรณี input ผิด (422) หลังเปิด TCP_NODELAY	50 ms	2 ms

แถว Windows กับ Docker วัดคนละสภาพแวดล้อม จึงเทียบตัวเลขข้ามแถวไม่ได้ นอกจากนี้ endpoint ที่เรียกโมเดลเป็น async และ SLO test ปิด connection หลังทุก request เพราะการจองแต่ละรายการมาจากลูกค้าคนละราย

การกัน Training-Serving Skew

- โค้ดชุดเดียว: train.py, calibrate.py และ serving/app.py เรียก build_features() ตัวเดียวกัน และอ่านรายชื่อคอลัมน์จาก configs/columns.yaml ไฟล์เดียว
- ตรวจ input ที่ API: schemas.py ตอบ 422 พร้อมบอกช่องที่ผิด เช่น lead_time ติดลบ ผู้เข้าพักเป็น 0 เดือนผิด
- ยืนยันแล้ว: ทำนายด้วยมือและผ่าน API ได้ p_cancel = 0.6886 เท่ากัน และบังคับ thread เดียวแล้วผลไม่เปลี่ยน

10. Monitoring และ Retrain

ระบบตรวจ 4 ด้านทุกสัปดาห์ และเมื่อพบ drift จะเทรนใหม่บนหน้าต่างข้อมูลล่าสุด ในการจำลอง 8 สัปดาห์ด้วยตัวตรวจจริง retrain 2 ครั้ง (สัปดาห์ 1 และ 5)

ประเภท	ตรวจด้วย	ต้องใช้ label	เกณฑ์แจ้งเตือน
Data drift	Evidently DataDriftPreset + NannyML univariate (11 ฟีเจอร์)	ไม่ต้อง	NannyML พบ drift alert ตั้งแต่ 1 จุดขึ้นไป

ประเภท	ตรวจด้วย	ต้องใช้ label	เกณฑ์แจ้งเตือน
Prior shift	อัตรายกเลิกจริงเทียบ train (0.358)	ต้อง (มาช้า)	ต่างเกิน 0.05
Concept drift	PR-AUC / ECE บน สัปดาห์ที่ผ่านไปแล้ว	ต้อง (มาช้า)	PR-AUC ตกเกิน 0.03 หรือ ECE เกิน 0.05
สถานะระบบ	/metrics + Locust	ไม่ต้อง	error เกิน 1% หรือ p95 เกิน 350 ms

label มาช้า รู้ว่ายกเลิกจริงก็ต่อเมื่อถึงวันเข้าพัก prior shift และ concept drift จึงตรวจได้เฉพาะสัปดาห์ที่ผ่านไปแล้ว ส่วน data drift ตรวจได้ทันที

ที่พบในข้อมูลจริง อัตรายกเลิกรายสัปดาห์ของ production สัปดาห์ 1–5 อยู่ที่ 0.42–0.47 สูงกว่า train (0.36) คือ prior shift จริง ส่วนตัวตรวจ src/monitoring/check_drift.py พบNannyML พบ 3–8 alert ทุกสัปดาห์ใน 8 สัปดาห์แรก รายงาน Evidently รายสัปดาห์อยู่ใน reports/monitoring/drift/

การแยกชนิดของ drift ตัวตรวจบันทึกสาเหตุแยกตามชนิดไว้ในช่อง reasons ของ reports/monitoring/week_XX.json และ simulation_summary.csv ทำให้ดูได้ว่าสัญญาณมาจากข้อมูลเข้า อัตรายกเลิก หรือคุณภาพการทำนาย

สัปดาห์	สัญญาณที่พบ	อ่านผลได้ว่า
1	NannyML 4 alert + อัตรายกเลิกต่างจาก train เกิน 5% แต่ performance ไม่ตก	data drift + prior shift ยังไม่กระทบคุณภาพ
3	เพิ่ม ECE 0.0565 เกิน 0.05	เริ่มกระทบ calibration
14	PR-AUC ตกจาก baseline 0.176	performance drift (ตัวแทน concept drift)
16	PR-AUC ตก 0.216 และ ECE 0.0867	performance drift รุนแรง

นโยบายการเทรนใหม่

retrain เมื่อมีสัญญาณอย่างน้อย 1 ข้อจากตาราง และผ่าน cooldown 4 สัปดาห์นับจากการ retrain ครั้งก่อน ผล retrain จะถูก promote ก็ต่อเมื่อผ่าน gate ทั้ง 3 ข้อ (ECE $\leq$ 0.05, PR-AUC ตกจาก champion ไม่เกิน 0.03, ถ้าจำลองไม่ต่ำกว่า champion)

วงจร retrain (pipeline/production_sim.py)

- แบ่ง production เป็นรายสัปดาห์ตามวันเข้าพัก ตรวจ drift ทีละสัปดาห์
- พบสัญญาณและผ่าน cooldown $\rightarrow$ สร้างหน้าต่างข้อมูลล่าสุด 12 / 6 / 3 เดือน (train / validation / test)
- เทรน + calibrate ด้วย calibrate.py เดิม $\rightarrow$ register เป็น candidate

4. gate วัด candidate กับ champion บน test ของหน้าต่งใหม่
5. ผ่าน → promote → export → restart API / ไม่ผ่าน → ใช้ champion เดิม

สัปดาห์	ระยะเวลา	PR-AUC	ECE	กำไรจำลอง	Gate
1	2017-01-08 → 04-08	0.7909 → 0.7792 (ลด 1.48%)	0.0497 → 0.0464 (ลด 6.64%)	1,058,508 → 1,056,941 (ลด 0.15%)	ไม่ผ่าน Keep champion
2	2017-02-06 → 05-06	0.7989 → 0.7943 (ลด 0.58%)	0.0383 → 0.0358 (ลด 6.53%)	1,378,413 → 1,379,884 (เพิ่ม 0.11%)	ผ่าน Promote + Redeploy

ผลการรัน อยู่ที่ reports/monitoring/gate_YYYY-MM-DD.json และ simulation_summary.csv โดย 8 สัปดาห์ใช้เวลาราว 8 นาที (ไม่นับ Docker build) ผลอยู่ที่ gate_2017-04-08.json และ gate_2017-05-06.json

หลักฐาน gate ไม่ผ่าน

ทดลองปรับเกณฑ์ ECE เข้มเป็น ≤ 0.03 เพื่อสาธิต candidate version 2 ที่ ECE 0.0374 ถูกปฏิเสธและระบบใช้ champion เดิมต่อ (reports/monitoring/gate_2017-05-14_fail.json)

วงจรมีใช้ตัวตรวจจริง src/monitoring/check_drift.py ผ่าน --drift-cmd ตัวตรวจส่งสัญญาณทุกสัปดาห์ cooldown 4 สัปดาห์จึงจำกัดให้ retrain จริง 2 ครั้ง ถ้าไม่มี cooldown ระบบจะ retrain ทั้ง 8 สัปดาห์

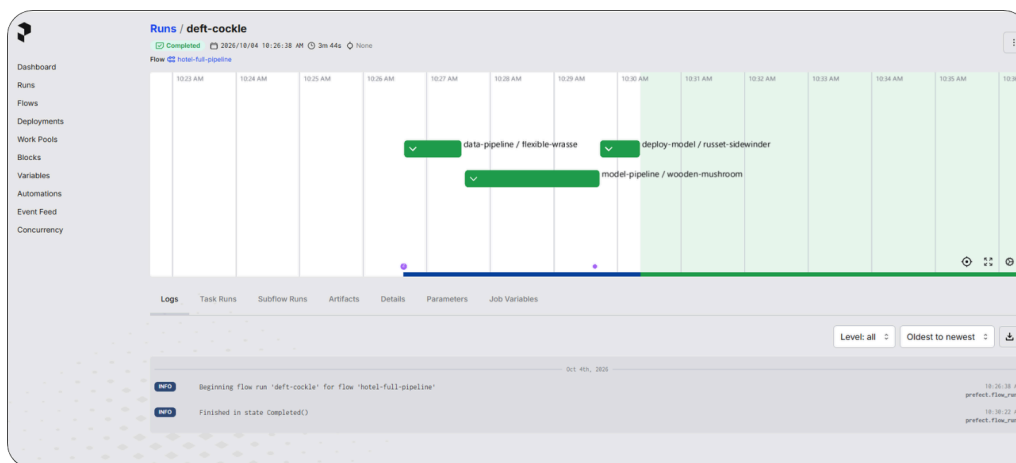
11. Pipeline และ CI/CD

ทั้งระบบรันด้วยคำสั่งเดียว python -m pipeline.run จบในเวลา 3 นาที 44 วินาที (หลัง build image ครั้งแรก) ขึ้นใหม่ทั้งสายหยุด ไม่มีโมเดลที่ไม่ผ่านด่านไปถึง API

ลำดับใน Prefect DAG (pipeline/flow.py)

ตรวจไฟล์ดิบ → SHA-256 → validate ข้อมูลดิบ (บันทึกผล) → clean → validate gate → split → สร้างข้อมูลเสีย → pytest → train → calibrate → export → docker compose up → รอ /health = ok แบ่งเป็น 3 subflow คือ data-pipeline, model-pipeline (train + calibrate) และ deploy-model (export → docker compose → health check)

คำสั่ง	ผล
<code>python -m pipeline.run</code>	รันตั้งแต่ข้อมูลดิบจน API พร้อมใช้
<code>python -m pipeline.run --demo-bad-data</code>	validate gate จับได้ 7 ข้อผิดพลาด หยุดภายใน 4 วินาที โมเดลไม่ถูกเทรน
<code>python -m pipeline.run --promote / --rollback</code>	สลับ alias และ redeploy API
<code>python -m pipeline.run --simulate --weeks 8 --retrain-on-drift</code>	จำลอง production รายสัปดาห์และวงจร retrain ใช้ตัวตรวจจริง src/monitoring/check_drift.py ผ่าน --drift-cmd (รูปแบบคำสั่งอยู่ใน README) ".venv-monitor\Scripts\python.exe src/monitoring/check_drift.py {week_csv} {out_json}"
<code>docker compose up</code>	เปิด API พร้อม healthcheck



หน้า Prefect UI ของการรัน `python -m pipeline.run` รันเต็ม 3 subflow ใน 3 นาที 44 วินาที



รัน --demo-bad-data flow ล้มที่ validate bad data (gate) ภายใน 4 วินาที

CI (.github/workflows/ci.yml) GitHub Actions รันทุก push เข้า main บน Python 3.13)

Extended validation: ภายหลังได้ลองทดสอบ production simulation เพิ่มเป็น 22 สัปดาห์ด้วย เพื่อประเมินความต่อเนื่องของ monitoring และ retraining pipeline

ด้าน	สิ่งที่ตรวจ	หลักฐานผ่าน	หลักฐานไม่ผ่าน
คุณภาพโค้ด	ruff check . และ pytest ของ API, clean, split และขั้นตัดสินใจ overbooking	https://github.com/Nipoon-T/MLOps-Predict-hotel-booking-cancellations-and-recommend-an-overbooking-level/actions/runs/37185922385/job/11138767865 6	https://github.com/Nipoon-T/MLOps-Predict-hotel-booking-cancellations-and-recommend-an-overbooking-level/actions/runs/37173784278/job/11135204576 8
ความถูกต้องของข้อมูล	tests/test_pandera_schema.py ตรวจ sample ผ่าน และข้อมูลเสียถูก reject ด้วย pytest.raises(ValidationError)	https://github.com/Nipoon-T/MLOps-Predict-hotel-booking-cancellations-and-recommend-an-overbooking-level/actions/runs/37185922385/job/11138767865 6	https://github.com/Nipoon-T/MLOps-Predict-hotel-booking-cancellations-and-recommend-an-overbooking-level/actions/runs/36968253784/job/11071659595 2
เกณฑ์คุณภาพโมเดล	gate ใน pipeline/retrain.py (ECE, PR-AUC ตก, ถ้าไร)	https://github.com/Nipoon-T/MLOps-Predict-hotel-booking-cancellations-and-recommend-an-overbooking-level/blob/main/re	https://github.com/Nipoon-T/MLOps-Predict-hotel-booking-cancellations-and-recommend-an-overbooking-level/blob/main/re

ด้าน	สิ่งที่ตรวจ	หลักฐานผ่าน	หลักฐานไม่ผ่าน
		ports/monitoring/gate_2017-05-13.json	ports/monitoring/gate_2017-05-14 fail.json

12. ข้อจำกัด การพัฒนาต่อ และการใช้ AI

ข้อจำกัด

- ถ้าใครเป็นการจำลอง: ความจุห้องและค่าชดเชย k เป็นค่าที่ตั้งเอง เพราะชุดข้อมูลไม่มีค่าจริง และไม่รู้ว่าลูกค้าที่ถูกปฏิเสธจะทำอะไร
- การเรียงลำดับการจองใช้ lead_time แทนวันที่จองจริง เพราะชุดข้อมูลไม่มี booking_date
- เลือก hyperparameter จาก ECE บน test set: ผลบน test จึงอาจดีกว่าความจริงเล็กน้อย ควรแยกชุดสำหรับเลือก hyperparameter ออกจากชุดที่ใช้รายงานผล
- deposit_type = Non Refund ยกเลิก 99% เป็นไปได้ว่าเป็นความผิดปกติเฉพาะของข้อมูลชุดนี้ ไม่ควรตีความเป็นพฤติกรรมลูกค้าทั่วไป
- ตัวเลข SLO วัดบน laptop เครื่องเดียว ที่รัน Locust ร่วมกัน p95 แกว่งตามภาระเครื่อง
- /metrics นับแยกต่อ worker ภาพรวมต้องดูจาก Locust หรือเครื่องมืออย่าง Prometheus
- already_occupied ต้องมาจากระบบจองของโรงแรม เวอร์ชันนี้ผู้เรียกต้องส่งค่าเอง
- ตัวตรวจ drift ดูฟีเจอร์ 3 ตัวที่โมเดลไม่ได้ใช้ (booking_changes, required_car_parking_spaces, total_of_special_requests ถูกตัดเป็น leakage) แต่ไม่ได้ดูฟีเจอร์หลักอย่าง deposit_type และ market_segment
- monitoring ต้องใช้ environment แยก (Python 3.12) เพราะ NannyML ไม่รองรับ Python 3.13 ที่ระบบหลักใช้
- ข้อมูล production สัปดาห์ละราว 1,300 แถว NannyML แบ่ง chunk ได้น้อยและเตือนว่าผลอาจไม่เสถียร

การพัฒนาต่อ

- ให้ตรวจฟีเจอร์ชุดเดียวกับที่โมเดลใช้
- เพิ่มการวัดความแม่นยำที่เมื่อลูกค้าเช็กอินจริง
- ย้ายไปทดสอบบนเซิร์ฟเวอร์แยก และเก็บสถิติระบบรวมทุก worker
- เพิ่ม model gate เข้า GitHub Actions

การใช้ AI ช่วยทำงาน

ส่วนงาน	ไฟล์	ใช้ AI ทำอะไร	ทีมตรวจสอบอย่างไร
Data + Validation	src/validate.py, src/clean_data.py	ร่างโค้ดและแก้โค้ดที่ไม่ตรงกับผลลัพธ์ที่ทีมต้องการ	ตรวจสอบผลลัพธ์ของข้อมูลตรงกับความต้องการของทีม
Modeling + Calibration	src/modeling/	ร่างโครงสร้างโค้ด, ช่วยแก้ errors, ช่วยวิเคราะห์ข้อมูล, แนะนำพารามิเตอร์ที่เหมาะสม	รันเพื่อทดสอบ, ทีมตัดสินใจเองว่าจะเก็บฟีเจอร์ไว้หรือตัดออก, ลองรันกับค่าพารามิเตอร์หลาย ๆ ชุดแล้วเลือกเฉพาะชุดที่ผ่านเกณฑ์ $ECE \leq 0.05$ แล้วเลือกตัวที่ PR-AUC ดีที่สุดในกลุ่มนั้น

ส่วนงาน	ไฟล์	ใช้ AI ทำอะไร	ทีมตรวจสอบอย่างไร
Decision layer	overbooking_decision/	ใช้ช่วยร่างโค้ด Monte Carlo, นโยบาย 4 แบบ, backtest, sensitivity, backtest แบบนับลูกค้าพักหลายคืน และช่วยวิเคราะห์สาเหตุที่ o* ไม่ตอบสนองต่อ k และร่าง decision_notes.md	เทส 24 ข้อผ่าน, เทียบผล sequential กับ naive, ตรวจสอบว่าจำนวนลูกค้าที่ถูกย้ายลดลงเมื่อ k เพิ่มขึ้น, รันบนข้อมูลจริงเอง
Serving + Load test	src/serving/, load_test/	ร่างโค้ด FastAPI และ schema ตรวจสอบ input, ร่างเทส API 18 ข้อ, ร่าง Locust scenario และสคริปต์วัดเวลาที่ละขั้น, ช่วยวิเคราะห์ผลวัดเพื่อหาสาเหตุที่ช้า 3 เรื่อง (n_jobs=-1, TCP_NODELAY, thread แย่ง GIL)	รันเทส 18 ข้อด้วยตัวเอง, วัด load test ข้างใน Docker หลังแก้ที่ละจุดแล้วเทียบก่อน-หลัง, เทียบ p_cancel ที่ทำนายด้วยมือกับผ่าน API ได้ 0.6886 ตรงกัน, ทดสอบ clone repo ใหม่บนเครื่องเปล่าแล้วรันตาม README, ทีมร่วมกันกำหนด SLO จากผลวัดจริง
Pipeline + Registry	pipeline/, docker-compose.yml, requirements-pipeline.txt	ใช้ AI ช่วยออกแบบและเขียนโค้ดในส่วนของ pipeline และช่วยแก้ปัญหาที่เจอเวลา error อย่างเช่นพวก version ของแต่ละเครื่องมือที่ไม่ตรงกัน	รันทุกขั้นตอนก่อนแล้วเทียบกับผลจริงของสมาชิกว่าตรงกันหรือไม่ แล้วนำมาสร้าง pipeline บนเครื่องจริงและตรวจสอบใน log จาก Prefect UI แล้วตรวจสอบพวก promote/rollback และทดสอบกับข้อมูลที่ไม่ตรงเกณฑ์ว่าถ้าเจอแล้วว่า pipeline ต้องหยุด
Monitoring + CI/CD	.github/workflows/ src/monitoring/check_drift.py data/production_weeks/ pipeline/production_sim.py pipeline/retrain.py reports/monitoring/	อธิบายการทำงานของโค้ด ออกแบบโค้ดให้เข้าใจง่าย ใช้และทำงานได้จริง และช่วยแก้ปัญหา error และ debug	รันทดสอบทุกขั้นตอน ตรวจสอบผลลัพธ์ว่าตรงกับแผนงานที่วางไว้จริงหรือไม่

วิธีรันซ้ำ ติดตั้ง Python 3.13 และ Docker Desktop ดาวน์โหลด hotel_bookings.csv จาก Google Drive ของทีมไปที่ data/raw/ แล้วรัน python -m pipeline.run บน Windows ต้องเปิด Long Path ก่อนติดตั้ง Prefect รายละเอียดอยู่ใน README.md สร้าง venv แล้ว pip install -r requirements-pipeline.txt และสร้าง .venv-monitor (Python 3.12) จาก requirements-monitoring.txt สำหรับตัวตรวจ drift